

RHU LabChain

System workflow guide

From patient registration to released laboratory records: a detailed guide to the seven workspaces, laboratory handoffs, version controls, encrypted publication, and public verification.

System status at delivery

The native development stack was stopped on 05 September 2026 (Asia/Singapore). The web server on port 5173 and services on ports 3001-3004 are no longer listening. Saved project data was retained. This PDF can be read while the application is off.

How to use this guide. Each phase explains the responsible role, inputs, actions, outputs, and handoff. Notes marked **Current interface limitation** identify where the existing screens do not expose every service operation. A service capability is not automatically a completed browser workflow.

Pages	Contents
2-3	System architecture and role responsibilities
4-6	Access, registration, consultation, payment, specimens, results, and QC
7-8	Submission, independent approval, encryption, proof, and release
9-10	Doctor/patient access, public verification, and status meanings
11-12	Exceptions, offline behavior, and a worked synthetic example
13	Shutdown/restart instructions, boundaries, and source references

Scope: project version 0.1.0, including the role-specific interfaces and neumorphic styling present on the preparation date. The system is a Phase One academic prototype for synthetic or formally de-identified records. This document explains software behavior, not clinical interpretation or a validated clinical procedure.

Source: README.md; apps/web/src/auth/workspaces.js; local process and port verification.

How the system fits together

The interface sends requests through /api paths to four services. Identity, laboratory workflow, encrypted packages, and authenticity proofs have separate ownership. Services enforce the actual permissions.

Component	Responsibility	Port
Web interface	Role workspaces, forms, queues, personal reports, and public verification pages.	5173
Authentication	Accounts, passwords, roles, access/refresh sessions, account state, and authentication audit.	3001
Records & Sync	Patients, encounters, orders, specimens, result versions, QC, approval/release, doctor review, workflow audit, and publication outbox.	3002
Encrypted Storage	Canonical packaging, AES-256-GCM encryption, wrapped data keys, object storage, and integrity-checked retrieval.	3003
Verification	Opaque proof registration, version/release history, verification tokens, and redacted public authenticity responses.	3004

Three different records of the work

The **workflow record** describes the encounter and laboratory progress. The **encrypted package** captures an approved version for protected storage. The **proof** identifies an opaque record/version, encrypted-envelope hash, issuer, and relevant events. It is not the full patient report.

What was running locally

Native startup uses file-backed service data, filesystem object storage, and a file ledger under data/. Package encryption is implemented, but the object-store and ledger adapters are development simulations. Native mode does not run MySQL, private IPFS, or Hyperledger Fabric.

Alternative infrastructure mode

Docker mode exposes a gateway on port 8080 and supports MySQL plus private Kubo/IPFS. Fabric requires a separate network bootstrap and configured identities. A /ready response must confirm the adapter before describing the running system as real Fabric/IPFS infrastructure.

Source: docs/ARCHITECTURE.md; scripts/dev-local.mjs; README.md; storage crypto/envelope.js.

Seven roles, seven workspaces

Login opens the assigned role workspace. A user with several assigned roles can select an active workspace in the sidebar. This focuses navigation and actions; backend checks still evaluate the authenticated account.

Role / default username	Main responsibility	Handoff
Administrator admin@lab.local	Accounts, roles, audit, health, sync, and backup guidance. No implicit clinical superuser access.	Staff access and available services.
Registration / cashier cashier@lab.local	Register/find patients, classify payment, and track requests.	Identity to doctor; classified request to lab.
Doctor doctor@lab.local	Consultations, signed lab requests, and released-result reviews for authorized encounters.	Signed request; later interpretation and follow-up.
Laboratory staff labstaff@lab.local	Specimens, result entry, QC, repeats/referrals, and submission.	QC-ready result version to supervisor.
Laboratory supervisor supervisor@lab.local	Review, approve/reject, release, and inspect workflow audit.	Released report to assigned doctor and linked patient.
Patient patient@lab.local	Own released reports, history, and verification QR.	May share public verification link.
Public verifier verifier@lab.local	Check a report link/token for redacted authenticity information.	Proof/status information only.

Access is assigned, not chosen at login

Users sign in with email addresses. The switcher offers only already-assigned roles. Unrelated navigation and creation buttons are hidden; restricted routes redirect to the active workspace. Only the cashier has the patient-registration button, and only the doctor has request creation.

The patient lands directly on personal released results. The verifier receives a focused link/token form. Staff home screens show relevant tasks and workload; infrastructure health is concentrated in administration.

Source: auth-service/src/constants.js; web/src/auth/workspaces.js; App.jsx. Default seed usernames shown; passwords excluded.

Start an encounter and request tests

1. Sign in

Owner: every authenticated user. Enter the assigned email and password. Authentication checks credentials/account status and returns tokens and roles. Seed accounts must change their temporary password before protected workflows. Signing out clears the application session.

2. Register or locate the patient

Owner: cashier. Open Patients and search by name/code before creating another record. Register first/middle/last name, suffix if applicable, birth date, sex, contact number, email, and address. The server creates a patient record and audit event. Retain the patient ID for the encounter.

Patient-account linking: a login account and a patient record are separate entities. The account must be linked to the correct record for personal results to work. The backend provides account-linking endpoints; the current registration screen does not expose that operation.

3. Document the consultation

Owner: doctor. Open Consultations and enter patient ID, optional visit ID, chief complaint, clinical notes, assessment, and plan. With no visit ID, the service can create an outpatient visit. An explicitly supplied visit must belong to the patient.

4. Sign the consultation and create the order

The consulting doctor must complete the consultation before ordering. The request identifies consultation/patient, priority (Routine, Urgent, STAT), clinical reason, panel codes, and optional test codes. Catalog panels expand into required tests. The server records the doctor/signing time, creates an opaque record identifier, and sets **REQUESTED**.

Current interface limitation: consultation completion

The Consultation page saves a draft but has no Complete/sign action. The backend requires POST `/api/records/consultations/{id}/complete` before order creation. An authorized API client is currently needed. Clicking Create request alone does not sign a draft consultation.

Handoff: the cashier receives the requested order. Unsigned consultations, unknown catalog codes, patient mismatches, or a different consulting doctor block order creation.

Source: PatientsPage.jsx; ClinicalPages.jsx; RecordsService.createConsultation, completeConsultation, createOrder; records-service/src/app.js.

Payment and specimen handling

5. Classify payment

Owner: cashier. Open Payments, select a requested order, and choose Paid, Free, Waived, or Subsidized. Paid uses an amount and receipt/authorization reference. Free/Waived requires an administrative reason in Remarks. Subsidized records the program. Classification is administrative; it does not edit test values.

Output: an audited classification and **PAYMENT_CLASSIFIED**. The request becomes eligible for accession. Invalid or out-of-sequence commands are rejected by the service.

6. Accession the specimen

Owner: laboratory staff. Open Specimens. Eligible states include **PAYMENT_CLASSIFIED**, **ACCESSIONED**, and **RECOLLECTION_REQUIRED**. Record sample type and use or generate a specimen/barcode code. Accession creates a traceable specimen identity associated with the order.

7. Record collection

After accession, use the specimen action again to record collection time and remarks. The current screen selects accession for an unaccessioned order and collection for an **ACCESSIONED** order. The sequence is **ACCESSIONED -> COLLECTED**.

Checkpoint	What must match
Identity	Patient, selected order, and specimen identity refer to the same encounter.
Required material	Specimen type supports the requested catalog tests.
Sequence	Payment classification precedes accession; accession precedes collection.
Traceability	Events retain actor/time and order association; recollection does not erase the original history.

Specimen exceptions

The service supports receipt and rejection operations and preserves rejected-specimen history. Rejection can require recollection. The current modal exposes accession/collection; receipt/rejection requires the corresponding authorized backend operation.

Handoff: the collected specimen and its order enter the testing workflow.

Source: WorkflowQueuePage.jsx; records-service/src/app.js; domain/statuses.js; specimen methods in RecordsService.

Testing, QC, repeat, and referral

8. Enter results

Owner: laboratory staff. Open Result entry for COLLECTED or IN_TESTING orders. Enter each test code/name, result value, optional numeric value, unit, applied reference range, and remarks. The service creates/uses an editable version and associates test attempts. Values belong to a particular version, not a freely editable final report.

9. Record quality control

Open Quality control and record pass/fail, control lot, observed control value, and remarks. Versions may pass through DRAFT, QC_PENDING, QC_PASSED, or QC_FAILED. Entering a measured value alone does not make a report ready for submission or release.

10. Resolve failures

Use Repeat & referral for applicable failed/repeat/referred work. A repeat records a reason and another attempt while retaining the failed attempt. A referral records the external laboratory, reason, and sent time. The external result must be received and incorporated before it satisfies the required item.

Path	Required outcome before submission
Normal test	Result item exists and its latest required attempt has passing QC.
Repeat	Repeat is completed/closed; the latest required attempt passes. Earlier failures remain recorded.
Referral	External result is incorporated with provenance. A sent referral alone remains unresolved.
Missing item	Complete every required item. Other passed tests do not bypass missing work.

Current interface limitation: exception completion

The interface can initiate repeat/referral work but does not expose the complete item-level repeat lifecycle or referral-result incorporation form. Backend operations, including POST `/api/records/referrals/{id}/results`, are currently needed for those completion steps.

Handoff: laboratory staff submits only complete, QC-ready work. Reference ranges and result flags are recorded software fields, not automated diagnoses.

Source: WorkflowQueuePage.jsx; result/QC/repeat/referral methods and submitVersion validation in RecordsService.

Submit, review, approve, or correct

11. Submit the version

Owner: laboratory staff. Open Submit results. The service verifies every required item has a passing latest attempt or incorporated referral, and required repeats are resolved. It captures a content hash, submitter, timestamp, and audit event, then sets **FOR_VERIFICATION**.

12. Review independently

Owner: supervisor. Open Supervisor review, inspect the submitted version, and choose Approve or Reject with remarks. Approval is allowed only for submitted results. **The submitter cannot approve the same version**, even when the account holds multiple roles.

Decision	System result
Approve	Version becomes VERIFIED; normal order advances through APPROVED to STORAGE_PENDING. A durable publication event starts encrypted storage/proof registration. This is not patient-visible release.
Reject	Submitted version is retained as REJECTED with a reason. A new draft is cloned; a normal order returns to IN_TESTING. Laboratory staff completes corrections and required checks.
Correct after approval/release	Create a linked new version through the correction endpoint. Retain prior content; complete testing/QC, submission, approval, publication, and release for the new version.

Current interface limitation: detailed review

The current supervisor queue/modal contains an order summary and decision controls, not a full result/QC inspection view. Inspect authorized version details through the backend before approving in a demonstration. Post-approval correction creation is also an API operation, not a dedicated browser form.

Version rule: a correction creates a new version linked to its source. Releasing the corrected version marks an earlier released version SUPERSEDED in Records. A doctor review is separate and does not modify the laboratory version.

Source: RecordsService.submitVersion, approveVersion, rejectVersion, createCorrection, #processReleaseEvent; WorkflowQueuePage.jsx.

Encrypt, register proof, and release

Records coordinates publication after approval. Each external step produces a receipt; failure is recorded instead of being treated as successful publication.

Step / owner	Action	Completion evidence
13. Encrypt Storage service	Canonicalize the approved snapshot. Generate fresh data key and nonce; encrypt with AES-256-GCM; wrap the data key with the configured master key. Store the encrypted envelope.	Receipt identifies opaque record/version, envelope hash, and protected object reference.
14. Register proof Verification service	Register opaque identity/version, envelope hash, issuer, and permitted proof metadata. Native mode uses a file ledger; configured Fabric mode uses the gateway.	Registration receipt; publication REGISTERED and normal order LEDGER_REGISTERED.
15. Request release Supervisor + Records	Select Result release. Backend requires verified version and completed ledger registration. Records creates a release event and requests ledger release.	Successful release receipt is required; selecting the action is not completion.
16. Complete release Records + Verification	Save release receipt/token, mark order/version RELEASED, record actor/time, and create report/release metadata.	Publication COMPLETE; assigned doctor and linked patient can access the release.

What the proof means

The registered hash supports integrity comparison for a specific encrypted package/version and its recorded issuer/history. It does not establish medical correctness, authorize patient-record access, or store the full report.

Release queue visibility is not release permission

The interface lists several publication states, including approved/stored work. The backend still blocks release until registration is complete. Appearing in the queue does not prove that every dependency succeeded.

Confidentiality: names/patient codes, contact details, clinical notes, measured values, passwords, and keys are excluded from the allowed proof payload. Public QR responses also exclude private storage references.

Source: docs/ARCHITECTURE.md; docs/BLOCKCHAIN.md; storage crypto/envelope.js; RecordsService publication/release methods.

Use and verify a released report

17. Assigned doctor reviews

Screen: Doctor review. The assigned doctor records acknowledgment/interpretation and follow-up. The backend checks assignment and appends a separate review event; it rejects a duplicate review by that doctor for the same release. The order stays RELEASED and the proof is unchanged.

Current interface limitation: the review queue offers order summary and review fields, not a full report viewer. Detailed result inspection currently requires an authorized backend view/API.

18. Patient views personal reports

Screens: My laboratory records, My history, Verification QR. The server returns released versions only for the patient record linked to the account. View report opens the report layout. Missing account links are rejected; no eligible releases produces an empty state.

Current interface limits: My history uses the same released-results source as the main portal, not a complete encounter timeline. Download appears only when a download URL exists; the current own-results response does not supply one. Verification QR selects the first report with a token rather than providing a report picker.

19. Public verifier checks authenticity

Paste a verification link or token in Verify a report. The interface opens `/verify/{token}` and the public endpoint returns redacted authenticity information. A QR link can also be opened directly without signing into a verifier account.

Public response can show	Public response must exclude
Opaque record reference, status, version, issuer, release information, and ledger mode.	Patient name/code, address/contact details, measured values, clinical notes, private storage location, credentials, and keys.

Implementation boundary

The patient portal currently reads released result data from Records; it does not demonstrate a complete encrypted-object download/decryption journey. Public verification is an authenticity response, not permission to view the report. Test revoked/superseded/error cases against the configured adapter.

Source: PatientPortalPage.jsx; PublicVerificationPage.jsx; DashboardPage.jsx; RecordsService.ownReleasedResults and recordDoctorReview.

Understand the status lifecycle

Order, result-version, and publication status answer different questions. A generic display label such as Completed does not prove a report is released.

Order status	Meaning / next responsible party
REQUESTED	Signed request exists; cashier classifies payment.
PAYMENT_CLASSIFIED	Classification complete; laboratory accessions specimen.
ACCESSIONED	Specimen identity recorded; laboratory collects.
COLLECTED	Collection recorded; laboratory begins testing.
IN_TESTING	Results/QC in progress; laboratory finishes required work.
FOR_VERIFICATION	Submitted result awaits independent supervisor review.
APPROVED / STORAGE_PENDING	Approval recorded; encrypted publication starts.
STORED / LEDGER_PENDING	Storage succeeded; proof registration pending.
LEDGER_REGISTERED	Registration complete; supervisor may request release.
RELEASED	Release receipt accepted; authorized access available.
RECOLLECTION_REQUIRED	Specimen exception requires a new collection path.
CANCELLED	Order is outside the normal progression.

Result-version lifecycle

DRAFT / QC_PENDING -> QC_PASSED -> FOR_VERIFICATION -> VERIFIED -> RELEASED.
 QC_FAILED, REPEAT_REQUIRED, and REFERRED describe unresolved testing branches. REJECTED retains a rejected submission; SUPERSEDED retains a prior release replaced by a correction. These are version states, not all order states.

Publication lifecycle

NOT_STARTED -> STORAGE_PENDING -> STORED -> LEDGER_PENDING -> REGISTERED -> RELEASE_PENDING -> COMPLETE. RETRY_WAIT and FAILED_REVIEW identify failed dependency work. Separate events/receipts track storage, registration, and release.

Source: records-service/src/domain/statuses.js; transitions and publication methods in RecordsService.

Exceptions, retries, and offline work

Situation	Expected handling
QC failure / missing test	Resolve required results, QC, repeats, or incorporated referral work. The backend prevents premature submission.
Supervisor rejection	Retain rejected version and reason; continue in the new draft. Do not overwrite the submitted version.
Storage/ledger unavailable	Keep failed publication event. Release remains blocked. Restore dependency and use an authorized retry/finalize operation.
Repeated publication failures	Implementation increases delay, capped at one hour. An event reaches FAILED_REVIEW after 10 failed attempts; investigate it.
Duplicate/lost response	Same idempotency key and input can replay the original result. Same key with different input produces a conflict.
Access rejected	Check assigned role, consultation/order doctor, and patient-account linkage. General role membership alone is insufficient.

Browser support currently present

Patient registration drafts and queued commands use IndexedDB. Offline sync lists local entries and can mark them for retry. It does not currently run a complete automatic replay loop. Reconnecting or clicking Schedule retry does not prove the command reached the server.

Server recovery currently present

Approval/release attempt dependent operations and persist failed outbox state. Authorized retry/finalize endpoints exist. The current Records startup code does not schedule a background outbox worker; unattended automatic recovery should not be claimed from the queue model alone.

Local data caveat

Logout clears saved drafts, but the current helper does not clear the IndexedDB outbox. Do not assume all queued local data is erased. Offline approval, release, role administration, and backup/restore are not supported browser workflows.

Project documentation describes a broader recovery design than the current UI completes. Queue state and backend receipts are the evidence that work was applied. Preserve failed work for review.

Source: `offline/indexedDb.js`; `OperationsPages.jsx`; `PatientsPage.jsx`; `RecordsService.#failOutbox`; `server.js`; `docs/OFFLINE_SYNC.md` (design context).

Worked example: one synthetic request

The identifiers and actions below are illustrative, not live patient data. This is the intended handoff sequence, including backend-only steps needed by the current interface.

Step	Actor and action	Evidence
1	Admin checks role assignments and accounts.	Distinct users; required password changes completed.
2	Cashier registers Demo Patient A and links the patient login using the account-link API.	Patient ID and correct account association.
3	Doctor creates consultation, completes it through the API, and requests catalog tests.	Signed consultation; REQUESTED order.
4	Cashier records payment classification and reference/reason.	PAYMENT_CLASSIFIED and payment event.
5	Lab staff accessions and collects specimen.	Unique specimen identity and collection timestamp.
6	Lab staff enters all required tests and QC; resolves repeat/referral exceptions.	Latest required attempts satisfy submission gates.
7	Lab staff submits; a different supervisor inspects details and approves.	FOR_VERIFICATION then VERIFIED; independent approver.
8	Storage encrypts; Verification registers proof.	Matching opaque record/version in storage and registration receipts.
9	Supervisor requests and completes release.	RELEASED / COMPLETE, timestamp, and token.
10	Doctor reviews; linked patient views report; verifier opens QR link.	Separate review, patient isolation, redacted proof response.

What to demonstrate

Show one normal case and one blocked transition, such as submission without passing QC or release without registration. Keep the order, version, actors, and status history consistent. A UI preview verifies layout, not execution of MySQL/IPFS/Fabric.

Completion condition: the release is available to the correct patient/doctor, the public response excludes clinical data, and relevant version, approval, storage, ledger, and release evidence is retained.

Source: Current service gates and interfaces; docs/TESTING.md. This example was not executed during PDF preparation.

Operations, limits, and references

Shutdown and restart

The native system is off. No listening sockets remained on ports 5173 and 3001-3004 after its process tree was terminated. Data directories, account records, and environment files were retained. Restart from the repository root with **npm run dev:local**, then open **http://localhost:5173**. In an interactive terminal, Ctrl+C stops the native stack.

Do not run `npm run seed` merely to restart: seeding resets existing development account passwords and forces another password change. `/health` checks liveness; `/ready` checks adapter/dependency readiness. Retained data is not a verified backup, and this shutdown did not create one.

Prototype boundary

The seven workspaces are implemented, but consultation signing, patient linking, full clinical review, exception completion, corrections, automatic offline replay, and report downloads have the interface gaps described here. Native storage/ledger adapters simulate infrastructure. This guide does not claim a new end-to-end or real-infrastructure test pass.

The documented backup scripts target Docker/MySQL/IPFS and need a verified recovery plan. Fabric deployment, coordinated recovery, production security/privacy validation, and institutional clinical approval remain separate work.

Repository reference	Purpose
README.md; docs/ARCHITECTURE.md	Startup, adapters, architecture, scope.
apps/web/src/auth/workspaces.js; App.jsx	Workspaces, active role, route gates.
apps/web/src/pages/*.jsx; offline/indexedDb.js	Actual forms/queues, report/QR behavior, offline limits.
services/auth-service/src/constants.js; scripts/seed.js	Role permissions and seed behavior.
services/records-service/src/app.js; domain/statuses.js; services/recordsService.js	Endpoint checks, lifecycle gates, corrections, publication, and access.
services/storage-service/src/crypto/envelope.js; docs/BLOCKCHAIN.md	Encryption and permitted proof fields.
docs/OFFLINE_SYNC.md; docs/TESTING.md	Recovery design and required testing evidence.

Source: Local repository inspected 05 September 2026. No passwords, live verification tokens, or private key material are included.